



南開大學
Nankai University

密码与网络空间安全学院
深度学习及应用（高阶课）实验报告

循环神经网络

姓名：谢小珂

学号：2310422

专业：物联网工程

2026 年 5 月 2 日

目录

1 引言	2
2 数据集、预处理与训练设置	2
2.1 数据集统计	2
2.2 输入表示、训练流程与评价指标	3
3 RNN 与 LSTM 原理及实现	3
3.1 基线 RNN 模型结构与实现	3
3.2 LSTM 模型结构与实现	4
3.3 模型参数规模比较	5
4 训练过程与整体指标分析	5
4.1 关键训练节点	5
4.2 最佳模型指标比较	6
5 逐类结果与预测矩阵分析	6
5.1 逐类 F1 变化	6
5.2 预测矩阵与误分类方向	7
6 LSTM 性能优于 RNN 的原因分析	8
7 实验结论	8

1 引言

循环神经网络适用于字符序列建模。姓名语言识别需将变长字符序列压缩为分类隐表示，不同语言的姓名在字符集合、前后缀及相邻字符组合上存在统计差异，因此需逐字符读取并整合序列信息。任务为 18 类语言分类，难点在于拉丁字符类别间相似度高，且小样本类别易受大类预测偏置影响，故仅凭准确率不足以评价模型，需结合逐类 F1 与预测矩阵。

实验构建 RNN 与 LSTM 两类模型，在统一数据划分、one-hot 编码、隐藏维度、批量大小、学习率、正则化及早停策略下训练，比较验证损失、准确率、macro-F1 及混淆矩阵，并分析 LSTM 性能差异。该设定确保差异主要源自循环单元结构本身，便于评估门控机制对序列建模的影响。后续分析基于原始结果文件。

2 数据集、预处理与训练设置

2.1 数据集统计

实验数据来自姓名分类任务，共 20074 条姓名、18 个语言类别、58 个字符特征。姓名长度最短为 2，最长为 19，平均长度为 7.1521。按照 0.15 的比例划分验证集后，训练集包含 17062 条样本，验证集包含 3012 条样本；随机种子固定为 2024。表1 汇总数据规模与训练配置，数值直接对应 dataset_summary.json 和 run_config.json。

数据项	数值	训练配置	数值
全部样本数	20074	hidden size	256
训练集样本数	17062	batch size	128
验证集样本数	3012	epochs 上限	100
类别数	18	early stopping patience	10
字符特征数	58	learning rate	0.001
姓名长度范围	2-19	weight decay	0.0001
平均姓名长度	7.1521	grad clip	5.0
验证比例	0.15	运行设备	CUDA

表 1: 数据集规模与训练配置

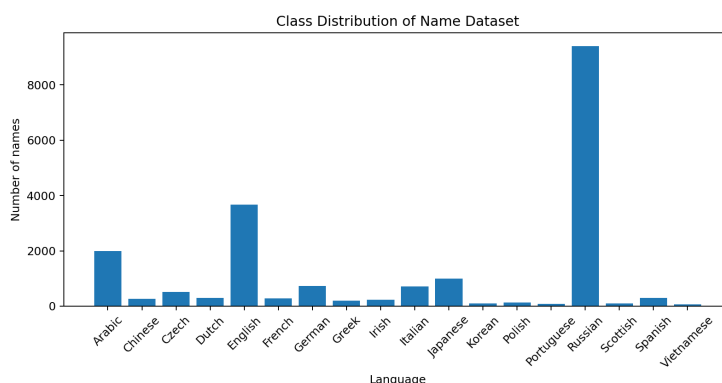


图 2.1: 姓名语言类别样本分布

图2.1 表明数据集存在明显长尾分布。Russian 为 9408 条，占全部样本约 46.86%；English 为 3668 条，占约 18.27%；Arabic 为 2000 条，占约 9.96%。相较之下，Vietnamese、Portuguese、Korean、Scottish 分别只有 73、74、94、100 条。验证集中 Russian 有 1411 条，English 有 550 条，而 Vietnamese 与 Portuguese 均只有 11 条，Korean 为 14 条。该分布使 accuracy 更容易受到大类影响，macro-F1、逐类 F1 和预测矩阵因此需要与整体准确率共同分析。

2.2 输入表示、训练流程与评价指标

字符级姓名识别将每个字符转换为 58 维 one-hot 向量。设一个姓名序列为 $x = (x_1, x_2, \dots, x_T)$ ，其中 T 为姓名长度，模型沿时间维读入字符向量并输出 18 维类别对数概率。训练过程使用负对数似然损失或等价的交叉熵形式，优化器学习率为 0.001，并采用 weight decay=0.0001 与梯度裁剪阈值 5.0。early stopping 的 patience 为 10，保存验证 loss 最小的模型。

训练与验证流程核心逻辑

```

1  for epoch in range(1, max_epochs + 1):
2      train_loss = train_one_epoch(model, train_loader, optimizer, criterion)
3      val_loss, val_acc, val_macro_f1 = evaluate(model, val_loader)
4      if val_loss < best_val_loss:
5          best_val_loss = val_loss
6          save_checkpoint(model)
7          wait = 0
8      else:
9          wait += 1
10     if wait >= patience:
11         break

```

实验报告采用四类指标描述模型表现。accuracy 衡量总体预测正确比例；macro precision、macro recall 与 macro-F1 对每个类别等权平均，能够反映小样本类别表现；loss 描述模型对验证标签的拟合程度；预测矩阵用于定位具体类别间的错误流向。由于数据分布不平衡，报告中不将 accuracy 单独作为判断依据，而是结合 macro-F1 和预测矩阵解释模型差异。

3 RNN 与 LSTM 原理及实现

3.1 基线 RNN 模型结构与实现

普通 RNN 在每个时间步将当前字符向量 x_t 与上一时刻隐藏状态 h_{t-1} 合并，得到当前隐藏状态：

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \quad \hat{y} = \text{LogSoftmax}(W_o h_T + b_o). \quad (1)$$

其中 h_T 表示模型读完整个姓名后的序列表示，分类层将其映射为 18 个语言类别的对数概率。实验中打印得到的基线 RNN 网络结构如下，输入维度 58 对应字符 one-hot 特征，输出维度 18 对应语言类别。

基线 RNN 网络结构

```

1  CharRNN(
2      (rnn): RNN(58, 256)
3      (h2o): Linear(in_features=256, out_features=18, bias=True)
4      (softmax): LogSoftmax(dim=1)
5  )

```

RNN 分类器核心实现

```

1  class CharRNN(nn.Module):
2      def __init__(self, input_size, hidden_size, output_size):
3          super().__init__()
4          self.rnn = nn.RNN(input_size, hidden_size)

```

```

5         self.h2o = nn.Linear(hidden_size, output_size)
6         self.softmax = nn.LogSoftmax(dim=1)
7
8     def forward(self, line_tensor):
9         rnn_out, hidden = self.rnn(line_tensor)
10        output = self.h2o(hidden[0])
11        return self.softmax(output)

```

该结构的计算路径较短，参数量和训练时间相对较低。对应限制也较直接：序列信息被压缩到单一隐藏状态中，若前部字符线索需要保留到末尾，普通 RNN 更容易受到后续字符覆盖影响。姓名分类中部分类别依赖后缀和字符组合，但不同语言存在相似的拉丁字符形态，因此普通 RNN 在 English、German、Russian 等类别之间容易产生混淆。

3.2 LSTM 模型结构与实现

LSTM 在隐藏状态之外引入细胞状态 c_t ，通过门控机制控制信息写入、保留和输出：

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (2)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (3)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t). \quad (4)$$

本实验的 LSTM 分类器保持与 RNN 一致的输入规模、隐藏规模和输出规模，并在分类层前加入 dropout=0.2。这样设置后，二者主要差异集中于循环单元结构，便于分析门控机制对姓名字符序列建模的影响。

LSTM 网络结构

```

1 CharLSTM(
2     (lstm): LSTM(58, 256)
3     (dropout): Dropout(p=0.2, inplace=False)
4     (h2o): Linear(in_features=256, out_features=18, bias=True)
5     (softmax): LogSoftmax(dim=1)
6 )

```

LSTM 分类器核心实现

```

1 class CharLSTM(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size, dropout=0.2):
3         super().__init__()
4         self.lstm = nn.LSTM(input_size, hidden_size)
5         self.dropout = nn.Dropout(dropout)
6         self.h2o = nn.Linear(hidden_size, output_size)
7         self.softmax = nn.LogSoftmax(dim=1)
8
9     def forward(self, line_tensor):
10        lstm_out, (hidden, cell) = self.lstm(line_tensor)
11        feature = self.dropout(hidden[-1])
12        return self.softmax(self.h2o(feature))

```

LSTM 的输入门决定当前字符信息写入比例，遗忘门决定旧细胞状态保留比例，输出门决定细胞状态向隐藏状态暴露的比例。对于姓名识别任务，该机制能够保留更长跨度的字符信息。例如，模型

不必在每个时间步完全覆盖旧隐藏状态，而是可以通过细胞状态将具有判别意义的字符模式传递到序列末尾。

3.3 模型参数规模比较

在输入维度、隐藏维度和输出类别数相同的条件下，RNN 与 LSTM 的参数数量存在明显差异。RNN 循环层包含输入到隐藏状态、隐藏状态到隐藏状态及偏置参数；LSTM 则需要为输入门、遗忘门、候选状态和输出门分别设置对应参数，因此循环层参数数量约为普通 RNN 的 4 倍。表2 给出两类模型的参数规模。

模型	循环层参数数量	分类层参数数量	总参数数量	相对 RNN 倍数
RNN	80896	4626	85522	1.00
LSTM	323584	4626	328210	3.84

表 2: RNN 与 LSTM 参数规模比较

由表2 可知，二者分类层参数完全相同，差异集中于循环层。LSTM 总参数数量为 328210，约为 RNN 的 3.84 倍。该结果说明，LSTM 的性能变化并非来自输出层结构差异，而主要来自门控循环单元带来的状态更新方式变化。参数数量增加也会带来一定训练开销，但在本实验中 LSTM 训练耗时仅比 RNN 增加约 2 秒，说明该规模下额外计算成本仍处于可接受范围。

4 训练过程与整体指标分析

4.1 关键训练节点

表3 列出训练曲线中的关键节点。RNN 第 1 轮验证 accuracy 为 62.12%，第 21 轮取得最小验证 loss 0.6423；LSTM 第 1 轮验证 accuracy 为 55.71%，第 22 轮取得最小验证 loss 0.5862。LSTM 起始阶段收敛较慢，但中后期验证 loss 更低，说明门控结构在获得充分训练后产生了更好的验证拟合效果。

模型	节点	epoch	train loss	val loss	val accuracy / macro-F1
RNN	初始轮次	1	1.5870	1.3010	62.12% / 0.1352
RNN	最小验证 loss	21	0.4344	0.6423	80.31% / 0.5010
RNN	最高 accuracy	26	0.3490	0.6500	80.35% / 0.4855
RNN	停止前末轮	31	0.2727	0.7000	79.91% / 0.4752
LSTM	初始轮次	1	1.7568	1.5145	55.71% / 0.0871
LSTM	最小验证 loss	22	0.3997	0.5862	82.07% / 0.4774
LSTM	最高 accuracy	24	0.3632	0.5960	82.14% / 0.4791
LSTM	最高 macro-F1	30	0.2699	0.6353	81.64% / 0.5130

表 3: 训练过程中的关键节点

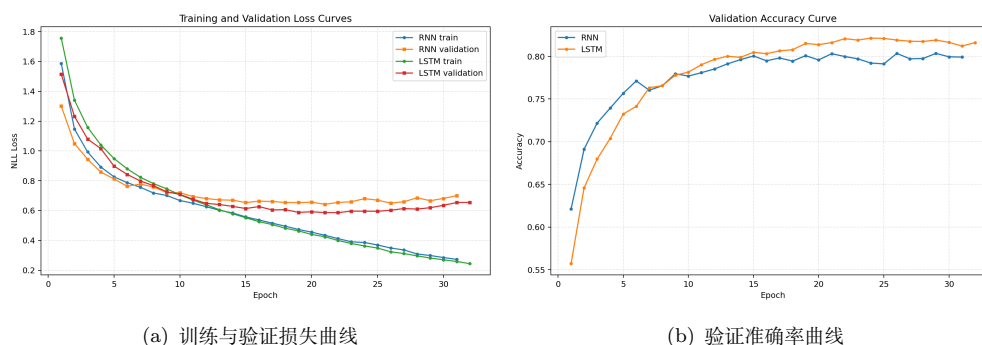


图 4.2: RNN 与 LSTM 的训练过程比较

图4.2 中，两类模型的训练损失均随 epoch 下降，说明优化过程有效。RNN 在第 21 轮取得最小验证损失，后续训练损失继续降低但验证损失上升至第 31 轮的 0.7000，表明模型已开始贴合训练集细节。LSTM 在第 22 轮取得最小验证损失 0.5862，之后验证损失同样波动上升，但准确率在 82% 左右保持相对稳定。该现象说明 early stopping 对当前任务是必要的，继续增加 epoch 不一定带来更优化结果。

4.2 最佳模型指标比较

表4 汇总以验证 loss 最小为准保存的最佳模型。LSTM 的验证 loss 为 0.5862，低于 RNN 的 0.6423；LSTM 的验证 accuracy 为 82.07%，高于 RNN 的 80.31%，提升 1.76 个百分点。LSTM 的训练耗时为 47.13 秒，RNN 为 45.13 秒，耗时增加约 2.00 秒。由此可知，在相同隐藏规模和相同验证集条件下，LSTM 以较小训练时间增加换取了更低验证损失与更高总体正确率。

模型	最佳 epoch	运行 epoch	val loss	accuracy	macro-P	macro-R	macro-F1
RNN	21	31	0.6423	80.31%	0.5491	0.4770	0.5010
LSTM	22	32	0.5862	82.07%	0.5645	0.4508	0.4774
差值	+1	+1	-0.0561	+1.76 pp	+0.0155	-0.0262	-0.0236

表 4: 验证集最佳模型指标

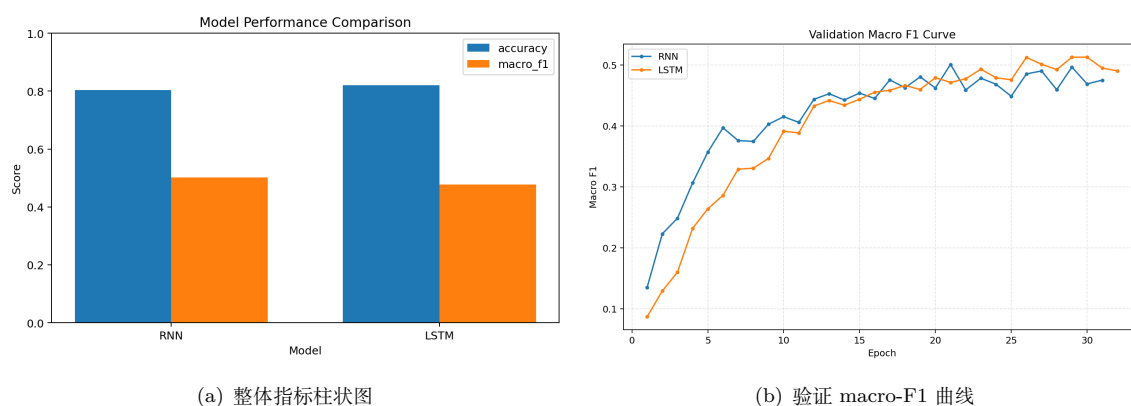


图 4.3: 模型指标对比与 macro-F1 变化

图4.3 反映出模型选择标准会影响结论细节。按最小验证 loss 保存时，LSTM 的 macro-F1 为 0.4774，低于 RNN 的 0.5010；按历史最高 macro-F1 观察，LSTM 在第 30 轮达到 0.5130，高于 RNN 的 0.5010。二者并不矛盾：验证 loss 更关注概率分布与真实标签的整体拟合，macro-F1 对各类别等权平均，小样本类别少量预测变化即可造成显著波动。当前结果可支持的结论是：LSTM 在验证 loss 和 accuracy 上优于 RNN，但小样本类别的稳定性仍存在改进空间。

5 逐类结果与预测矩阵分析

5.1 逐类 F1 变化

表5 给出 LSTM 相对 RNN 的逐类 F1 变化。LSTM 在 Dutch、Czech、French、English、Polish、Italian、Irish、Portuguese、Arabic、Russian 等类别上取得更高 F1，其中 English 由 0.7317 提升至 0.7744，Dutch 由 0.4533 提升至 0.5067。相较之下，Vietnamese 与 Korean 的 F1 下降较多；这两个类别的验证样本分别只有 11 条和 14 条，少量误判即可使 F1 接近 0。

类别	support	RNN-F1	LSTM-F1	$\Delta F1$	类别	support	RNN-F1	LSTM-F1	$\Delta F1$
Dutch	45	0.4533	0.5067	+0.0533	Spanish	45	0.4082	0.4000	-0.0082
Czech	78	0.3939	0.4460	+0.0521	Japanese	149	0.8293	0.8207	-0.0086
French	42	0.3500	0.4000	+0.0500	German	109	0.3804	0.3515	-0.0289
English	550	0.7317	0.7744	+0.0427	Chinese	40	0.6747	0.6337	-0.0410
Polish	21	0.3333	0.3704	+0.0370	Greek	30	0.7143	0.6545	-0.0597
Italian	106	0.6468	0.6784	+0.0316	Korean	14	0.1739	0.0000	-0.1739
Irish	35	0.4815	0.5098	+0.0283	Vietnamese	11	0.4444	0.0000	-0.4444
Portuguese	11	0.1176	0.1429	+0.0252	Scottish	15	0.0000	0.0000	0.0000
Arabic	300	0.9646	0.9788	+0.0142	Russian	1411	0.9201	0.9259	+0.0058

表 5: LSTM 相对 RNN 的逐类 F1 变化

逐类指标说明, LSTM 的优势主要体现在样本量中等或较大的类别上。Arabic、English、Italian、Russian 的 F1 均有提升, 其中 Arabic 达到 0.9788, Russian 达到 0.9259。小样本类别未必同步受益, 尤其是 Scottish、Korean、Vietnamese 等验证样本过少的类别, 其预测结果容易被 English、Russian 等大类吸收。该现象与第 2 节的长尾数据分布一致。

5.2 预测矩阵与误分类方向

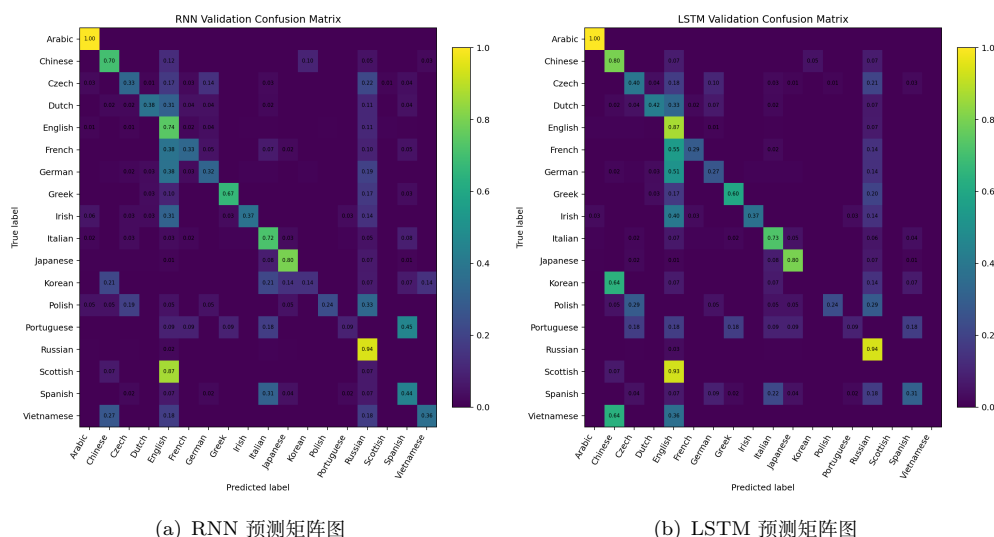


图 5.4: RNN 与 LSTM 在验证集上的预测矩阵

图5.4 显示, RNN 在验证集上正确预测 2419 条样本, LSTM 正确预测 2472 条样本, LSTM 多预测正确 53 条。表6 列出两个模型最主要的误分类方向。RNN 中 English $\rightarrow$ Russian 出现 62 次, LSTM 中该错误减少至 40 次; 与此对应, LSTM 的 English F1 从 0.7317 提升到 0.7744。German $\rightarrow$ English 在 RNN 中出现 41 次, 在 LSTM 中增加至 56 次, 说明 LSTM 对 English 的召回增强也带来了部分相近类别向 English 集中的问题。

RNN 真实类别	RNN 预测类别	次数	LSTM 真实类别	LSTM 预测类别	次数
English	Russian	62	German	English	56
German	English	41	Russian	English	42
Russian	English	31	English	Russian	40
German	Russian	21	French	English	23
English	German	21	Czech	Russian	16
Czech	Russian	17	German	Russian	15
French	English	16	Dutch	English	15
Spanish	Italian	14	Scottish	English	14

表 6: 验证集中高频误分类方向

高频误分类集中在字符形态相近或样本数量差异较大的类别之间。English、German、French、Irish、Scottish 均使用拉丁字符，且部分姓名后缀和拼写习惯具有重叠，模型容易将小类或中等样本类别归入 English。Russian 类样本量远高于其他类别，普通 RNN 更容易将 English 误判为 Russian；LSTM 缓解了这一错误，但并未完全消除类别不平衡造成的偏置。

6 LSTM 性能优于 RNN 的原因分析

从结构层面看，普通 RNN 依靠单一隐藏状态传递历史字符信息。每个时间步都会用当前字符更新 h_t ，早期字符信息需反复经过循环权重矩阵和非线性函数才能传到序列末端。姓名平均长度约为 7.15，最长达到 19；虽然该任务不属于长文本建模，但部分判别线索可能跨越姓名前后位置，例如前缀、后缀和连续字符组合。普通 RNN 缺少显式保留机制，因而在相近类别之间更容易产生混淆。

如第 3.2 节所述，LSTM 通过细胞状态以及输入门、遗忘门、输出门实现序列信息的选择性保留。对于姓名分类任务，该机制使模型不必在每个时间步用新字符完全覆盖旧表示，而可将早期前缀信息沿细胞状态传递至序列末端。实验数据支持这一解释：LSTM 的验证 loss 从 RNN 的 0.6423 降至 0.5862，accuracy 从 80.31% 提升至 82.07%，正确预测数量由 2419 条增加到 2472 条。

从类别结果看，LSTM 对大类和中等样本类别的改善更明显。English 的 F1 提升 0.0427，Dutch 提升 0.0533，Czech 提升 0.0521，French 提升 0.0500，Arabic 提升 0.0142，Russian 提升 0.0058。这些结果说明门控结构有助于捕捉姓名字符组合中的稳定模式。另一方面，按最小验证 loss 保存的 LSTM macro-F1 低于 RNN，Vietnamese 与 Korean 的 F1 下降至 0，说明 LSTM 的整体优势不等于所有类别同步提升。该结论与表5 和图4.3 一致。

结合参数规模也可解释两类模型差异。LSTM 的循环层参数量为 323584，显著高于 RNN 的 80896；增加的参数对应四组门控变换，使模型能够分别学习信息写入、遗忘和输出。该结构提高了模型表达能力，也更适合处理前后缀、字符组合和跨位置依赖。与此同时，LSTM 训练耗时仅由 45.13 秒增加至 47.13 秒，说明在当前任务规模下，额外门控计算未造成明显训练负担。

由此可知，LSTM 优于 RNN 的主要依据是验证 loss、accuracy 和正确预测数量；其原因在于门控结构对序列信息的选择性记忆。当前结果也显示，类别不平衡会限制 macro-F1，尤其是验证样本少于 20 的类别。后续可采用类别加权损失、分层采样或以 macro-F1 作为 early stopping 标准，使模型选择过程不只受大样本类别支配。

7 实验结论

本实验完成了 RNN 与 LSTM 在姓名语言识别任务上的训练与评估。基线 RNN 结构为 RNN(58,256) 接 Linear(256,18) 与 LogSoftmax；LSTM 结构为 LSTM(58,256)、Dropout(0.2)、Linear(256,18) 与 LogSoftmax。两类模型均在同一验证集上生成损失、准确率、macro-F1 曲线和混淆矩阵。

从验证结果看，RNN 最优验证损失为 0.6423，准确率为 80.31%，macro-F1 为 0.5010；LSTM 对应指标分别为 0.5862、82.07% 和 0.4774。以损失和准确率衡量，LSTM 表现更优；以各类别等权平均的 macro-F1 衡量，按最小损失保存的 LSTM 受小样本类别影响，数值低于 RNN。混淆矩阵显示，LSTM 将正确预测数从 2419 提升至 2472，English → Russian 的误判明显减少，但 German、French、Irish、Scottish 等类别仍倾向于被归为 English。

综上，LSTM 的门控机制能在字符级序列建模中更有效地保留判别模式，使验证损失和准确率得到改进；但数据长尾分布仍限制 macro-F1。后续优化可侧重类别不平衡处理、以 macro-F1 为导向的早停策略，以及小样本类别召回率提升。